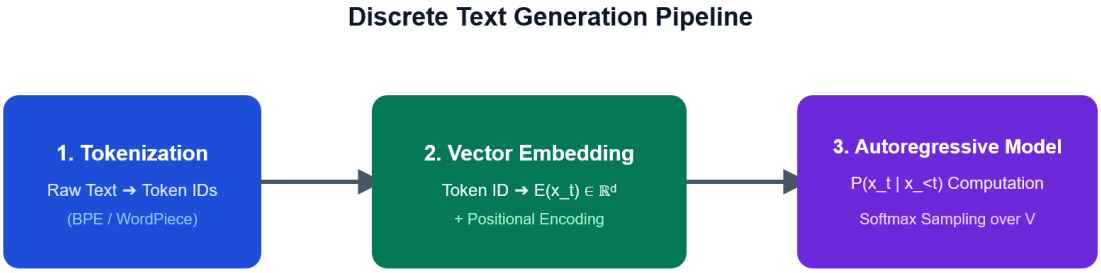
UNIT-III: Text Generation with Generative AI

1. Introduction to Text Generation

Text generation in Generative AI focuses on modelling the conditional probability distribution $P(X) = P(x_1, x_2, \dots, x_T)$ over sequence vectors of variable length T. Unlike image tensors, text is discrete, sequential, and non-differentiable in its raw token form.

Sequence Modeling Pipeline



A. Autoregressive Language Formulation

Text generators model joint probabilities by factorizing the sequence probability using the **Chain Rule of Probability**:

$$P(X) = P(x_1, x_2, \dots, x_T) = \prod_{t=1}^{T} P\left(x_t \mid x_1, x_2, \dots, x_{t-1}\right) = \prod_{t=1}^{T} P\left(x_t \mid x_{<t}\right)$$

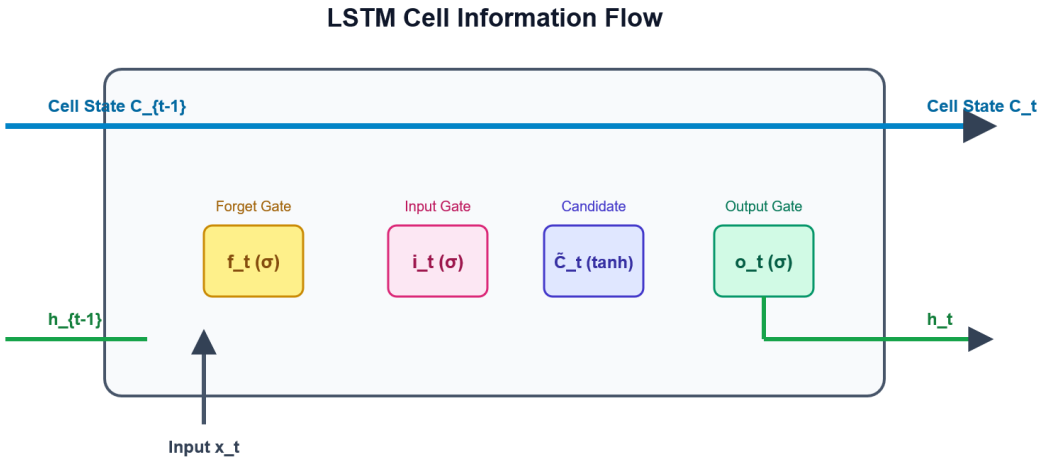
B. Discrete Text vs. Continuous Image Generation

- **Non-Differentiability:** Unlike pixel values in image generation (which are continuous real numbers), text consists of discrete indices sampled from a vocabulary $\mathcal{V}$. Because standard sampling operations are non-differentiable, backpropagation cannot pass through discrete token outputs directly (requiring cross-entropy loss over target token probability distributions).
- **Sequential Dependence:** The semantic value of a token x_t depends heavily on long-range temporal context $x_{<t}$.

2. LSTM-Based Text Generation

Long Short-Term Memory (LSTM) networks address the vanishing and exploding gradient problems of standard Recurrent Neural Networks (RNNs) by introducing a dedicated **Cell State** C_t alongside hidden states h_t.

LSTM Cell Architecture



A. Mathematical Gate Formulations

At timestep t, given input x_t and previous hidden state h_{t-1}:

1. **Forget Gate:** Decides what information to drop from the cell state:
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$
2. **Input Gate & Candidate Cell Update:** Decides which new values to update in the cell state:
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$
3. **Cell State Update:** Combines past state and new candidates:
$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$
4. **Output Gate & Hidden State:** Computes output filtered by cell state:
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(C_t)$$

Where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid activation function and $\odot$ is Hadamard (element-wise) multiplication.

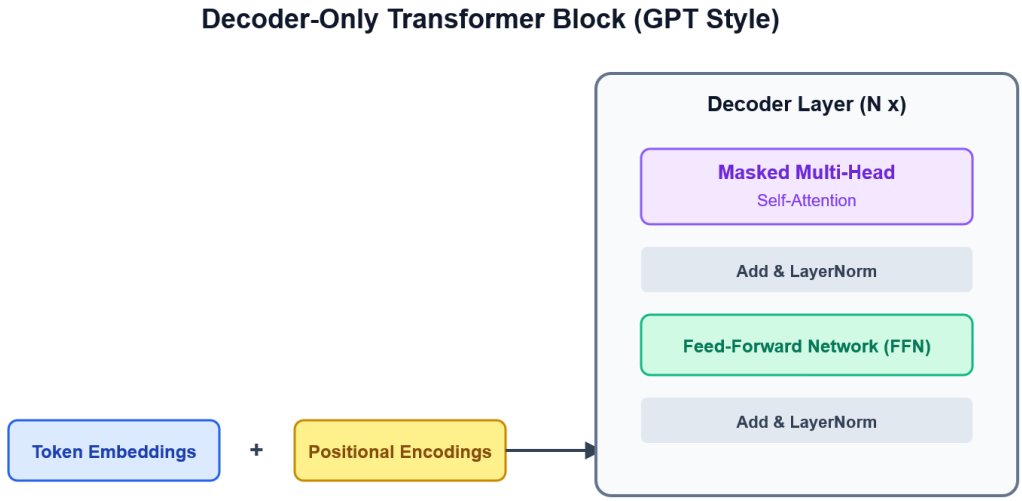
B. Teacher Forcing vs. Inference Decoding

- **Teacher Forcing (Training):** During training, the ground-truth target token x_t^* from the dataset is fed as input to step $t+1$, regardless of what model output $\hat{x}_t$ was generated. This stabilizes learning.
- **Autoregressive Decoding (Inference):** During generation, ground truth is unavailable. The model's own predicted token $\hat{x}_t \sim P(x_t \mid x_{<t})$ is fed back as the input for step $t+1$.

3. Transformer-Based Text Generation

Transformers (Vaswani et al., 2017) replace recurrent loops with **Self-Attention** and **Positional Encodings**, processing all sequence tokens simultaneously in parallel.

Transformer Decoder Architecture



A. Scaled Dot-Product Self-Attention

Self-attention maps Query (Q), Key (K), and Value (V) linear projections derived from input matrix $X \in \mathbb{R}^{T \times d_{\text{model}}}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- **Scaling Factor ($\sqrt{d_k}$):** Prevents large inner-product values in high dimensions from pushing softmax gradients into extremely small (saturating) regions.

- **Causal Attention Mask (M):** For text generation (Decoder-only models like GPT), future tokens must be hidden to preserve autoregressive properties. An upper-triangular matrix M is applied:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases}$$

B. Multi-Head Attention (MHA)

Allows the model to jointly attend to information from different representation subspaces at different positions:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

C. Positional Encoding

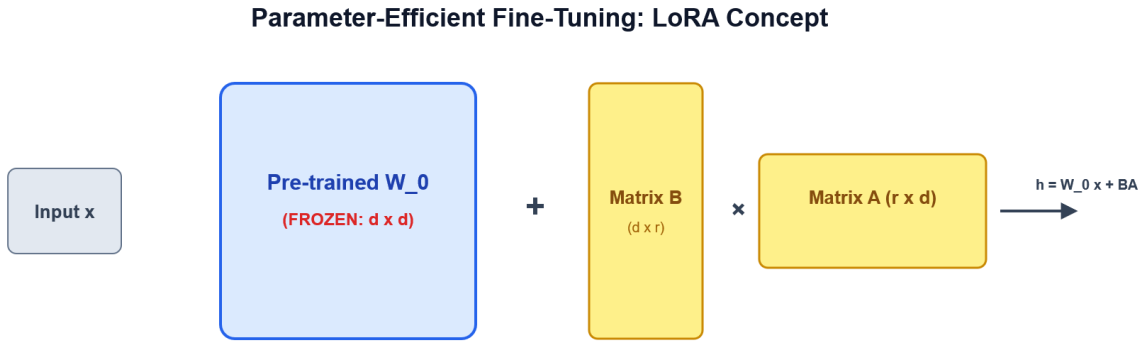
Because Transformers lack recurrence, sequence order must be injected using **Sinusoidal Positional Encodings** added directly to input embeddings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

4. Fine-Tuning Language Models

Pre-trained Base Language Models (P_{base}) are trained on raw text prediction. Fine-tuning adapts these models for downstream task execution and human preference alignment.

Fine-Tuning Pipeline & LoRA



A. Supervised Fine-Tuning (SFT)

SFT trains a base model on structured instruction-response pairs (x, y) using standard cross-entropy loss over response tokens y:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{i=1}^{|y|} \log P_\theta\left(y_i \mid x, y_{<i}\right)$$

B. Parameter-Efficient Fine-Tuning (PEFT / LoRA)

Full fine-tuning of billions of parameters is computationally expensive. **Low-Rank Adaptation (LoRA)** freezes the pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ and injects trainable rank decomposition matrices A and B:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}(B \cdot A)$$

Where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, and rank $r \ll \min(d, k)$. Only matrices A and B are updated during training.

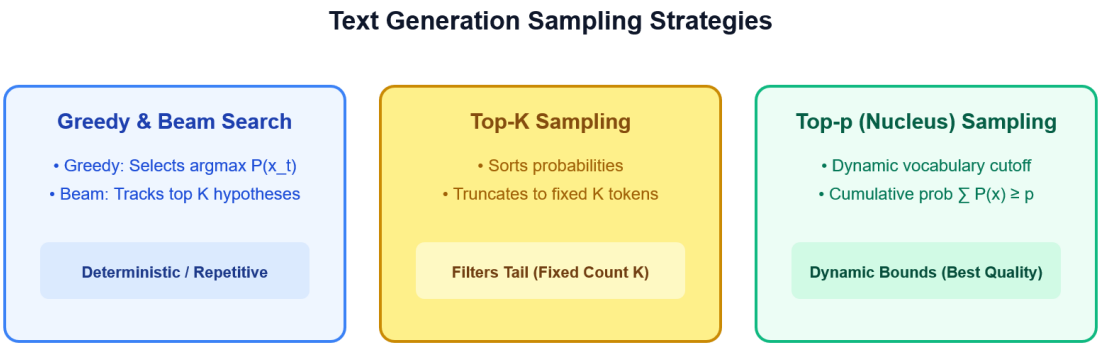
C. Alignment: RLHF & DPO

- **Reinforcement Learning from Human Feedback (RLHF):** Uses a learned Reward Model $R_\psi(x, y)$ and Proximal Policy Optimization (PPO) to maximize reward while penalizing divergence from the base model via KL Divergence:
$$\text{obj}(\theta) = \mathbb{E}_{(x,y)}\left[R_\psi(x, y)\right] - \beta\, D_{\text{KL}}\left(P_\theta(y \mid x) \,||\, P_{\text{base}}(y \mid x)\right)$$
- **Direct Preference Optimization (DPO):** Eliminates the need for a separate reward model by directly parameterizing the policy using preference pairs $(y_w \succ y_l \mid x)$:
$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{P_\theta(y_w \mid x)}{P_{\text{base}}(y_w \mid x)} - \beta \log \frac{P_\theta(y_l \mid x)}{P_{\text{base}}(y_l \mid x)}\right)\right]$$

5. Text Decoding Strategies & Sampling Techniques

During generation, the output logits z_i over vocabulary $\mathcal{V}$ are converted into probabilities using Softmax scaling with Temperature T:
$$P(x_i) = \frac{\exp(z_i/T)}{\sum_{j \in \mathcal{V}} \exp(z_j/T)}$$

Text Decoding Strategies



Decoding Methods Summary

- **Greedy Decoding:** Selects $x_t = \arg\max_{x \in \mathcal{V}} P(x \mid x_{<t})$ at every step. Often results in repetitive, low-diversity text.
- **Beam Search:** Maintains B (beam width) high-probability candidate sequences at each step, maximizing cumulative sequence likelihood:
$$\hat{X} = \arg\max_{X} \sum_{t=1}^{T} \log P\left(x_t \mid x_{<t}\right)$$
- **Temperature Scaling (T):** Controls randomness. Lower temperature ($T \to 0$) sharpens distribution toward greedy selection; higher temperature ($T > 1.0$) flattens distribution, increasing generation diversity.
- **Top-K Sampling:** Limits candidate generation by retaining only the K most probable tokens in $\mathcal{V}$, re-normalizing their probabilities before sampling.
- **Top-P (Nucleus) Sampling:** Dynamically selects the smallest subset of tokens $\mathcal{V}^{(p)} \subset \mathcal{V}$ whose cumulative probability mass exceeds threshold p:
$$\sum_{x \in \mathcal{V}^{(p)}} P\left(x \mid x_{<t}\right) \geq p$$

6. Text Generation Applications & Evaluation Metrics

A. Major Application Domains

1. **Machine Translation:** Translates source text across languages using Encoder-Decoder or Decoder-only models (e.g., NMT).
2. **Abstractive Text Summarization:** Generates novel summary sentences capturing key concepts rather than copying raw text extracts.

3. **Conversational Agents & Code Generation:** Powers interactive instruction-following chatbots (e.g., ChatGPT) and code synthesis assistants (e.g., Codex).

B. Evaluation Metrics

Metric	Primary Use Case	Core Mechanism	Formula / Key Principle
Perplexity (PPL)	Model Language Quality	Exponentiated average cross-entropy per token. Lower is better.	$\text{PPL}(X) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log P\left(x_t \mid x_{<t}\right)\right)$
BLEU Score	Machine Translation	Precision of n-grams with brevity penalty (BP).	$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$
ROUGE Score	Summarization	Recall-focused n-gram, longest common subsequence overlap.	$\text{ROUGE-L} = \frac{\text{LCS(Reference,Candidate)}}{\text{Length(Reference)}}$